Gateway către backend

Transmiterea citirilor și stocarea în SQLite

Echipă: Filipan Cristian Denis și Oana Rareș

Obiectiv. În CP02 gateway-ul primea date de la trei senzori virtuali. În CP03, gateway-ul trimite fiecare citire unui proces backend, care o salvează într-o bază de date. Testul a fost rulat pe Fedora, cu date sintetice.

Senzori virtuali CSV sintetic → **Gateway** TCP, port 5050 → **Backend** HTTP, port 8303 → **SQLite** Tabela citiri

Senzorii trimit JSON prin TCP. Gateway-ul adaugă câmpul `primit_la` și transmite fiecare pachet prin `POST /citiri`.

Confirmarea transmiterii

```
denis@fedora:~/Documents/CP03_irigare$ python3 demo.py
CP03 | Senzori -> Gateway -> Backend -> SQLite
[Backend] PID=6454 | HTTP 127.0.0.1:8303/citiri
[Backend] Baza SQLite: rulari/test-9xc_sppn/irigare.db
[Gateway] PID=6457 | TCP 127.0.0.1:5050
[Gateway] Astept senzori. Trimit la http://127.0.0.1:8303/citiri
Senzori: temperatura PID=6461, ploaie PID=6462, et0 PID=6463

Gateway: 72 confirmari HTTP 201. Exemple:
[Gateway] POST /citiri -> HTTP 201 | id=70 | temperatura nr=24
[Gateway] POST /citiri -> HTTP 201 | id=71 | ploaie nr=24
[Gateway] POST /citiri -> HTTP 201 | id=72 | et0 nr=24

Backend: 72 citiri stocate. Exemple:
[Backend] Stocat in SQLite | id=70 | temperatura nr=24
[Backend] Stocat in SQLite | id=71 | ploaie nr=24
[Backend] Stocat in SQLite | id=72 | et0 nr=24
```

Figura 1. Partea de sus a capturii de pe Fedora, după `python3 demo.py`. Gateway-ul afișează 72 de confirmări, iar backend-ul afișează 72 de citiri stocate.

Cum legăm cele două jurnale. Backend-ul validează câmpurile, inserează rândul în `citiri` și confirmă cu `HTTP 201` numai după ce salvarea a fost confirmată în SQLite (commit). ID-ul 70 apare atât la gateway, cât și la backend pentru mesajul 24 al senzorului de temperatură.

Verificarea stocării

Rezultatul testului: 72 din 72 de citiri

După ce termină cei trei senzori, demonstrația oprește gateway-ul și backend-ul. Scriptul `verifica.py` deschide apoi fișierul SQLite direct de pe disc, grupează citirile după senzor și le compară cu fișierul sursă.

```
Gateway si backend oprite. Citesc baza de pe disc.
Verificare: python3 verifica.py --db rulari/test-9xc_sppn/irigare.db
Baza SQLite: rulari/test-9xc_sppn/irigare.db
SELECT senzor, COUNT(*) FROM citiri GROUP BY senzor;
  temperatura: 24/24 citiri
  ploaie     : 24/24 citiri
  et0        : 24/24 citiri
OK: 72/72 citiri din baza corespund sursei.
```

Figura 2. Partea de jos a aceleiași capturi. După oprirea proceselor, SQLite conține câte 24 de citiri pentru temperatură, ploaie și ET_0. Mesajul OK confirmă că valorile, numerele mesajelor, unitățile și momentele simulate corespund sursei.

Exemplu de citire confirmată

ID SQLite	Senzor	Nr. mesaj	Ora simulată	Valoare
70	temperatura	24	23:00	18.5 °C

În rândul de mai sus, `timestamp` este ora 23:00 din scenariul sintetic, iar `primit_la` păstrează ora reală de recepție la gateway. Coloana `sursa` are valoarea `sintetic`.

Repetarea testului

Din directorul `CP03_irigare`, rulează:

```
python3 demo.py
```

Fiecare rulare își creează propria bază `irigare.db` și își păstrează jurnalele în `rulari/`. Datele stocate pot fi folosite ulterior pentru calculul necesarului de apă.

Referințe: http.client, http.server, sqlite3 din documentația Python. Figurile redau aceeași rulare pe Fedora.